

Chapter 1.3.1 — Why RNNs and LSTMs Were Not Enough

"To understand why the Transformer revolutionized Artificial Intelligence, we must first understand the limitations of the architectures that came before it."

Learning Objectives

By the end of this section, you will be able to:

- Understand how Recurrent Neural Networks (RNNs) process language.
 - Explain why sequential computation limits scalability.
 - Understand the Vanishing and Exploding Gradient problems.
 - Explain how Long Short-Term Memory (LSTM) networks improved upon RNNs.
 - Understand why even LSTMs struggle with long documents.
 - Explain why these limitations motivated the Transformer architecture.
-

1. Introduction

Natural language is inherently sequential. The meaning of a sentence depends not only on individual words but also on the order in which they appear.

Consider the sentence:

"The trophy didn't fit into the suitcase because **it** was too small."

What does **it** refer to?

Humans infer that **it** refers to the suitcase because we understand context.

Teaching machines to maintain this contextual understanding over long sequences was one of the biggest challenges in Natural Language Processing (NLP).

Before Transformers, the dominant architectures were:

- Recurrent Neural Networks (RNNs)
- Long Short-Term Memory Networks (LSTMs)
- Gated Recurrent Units (GRUs)

These models significantly advanced NLP but eventually reached practical limits that prevented them from scaling to today's Large Language Models.

2. Understanding Sequential Processing

Imagine reading a book.

You typically read:

Word 1 → Word 2 → Word 3 → Word 4

An RNN processes text in exactly this manner.

Input Sentence

I → Love → Artificial → Intelligence

Processing

Word1

|

▼

Hidden State 1

|

▼

Word2

|

▼

Hidden State 2

|

▼

Word3

|

▼

Hidden State 3

|

▼

Word4

Each hidden state depends on the previous hidden state.

Mathematically:

$$h_t = f(x_t, h_{t-1})$$

Where:

- x_t = current input token
- h_{t-1} = previous hidden state
- h_t = current hidden state

The hidden state acts as the network's memory.

3. Why Sequential Processing Is Slow

Suppose a sentence contains 2,000 words.

An RNN **cannot process word 1,500 until it has processed words 1 through 1,499.**

This dependency makes training inherently sequential.

```
Word1
  ↓
Word2
  ↓
Word3
  ↓
Word4
  ↓
...
Word2000
```

No matter how many GPUs you have, the model cannot fully parallelize this computation.

Production Impact

For modern datasets containing billions of tokens:

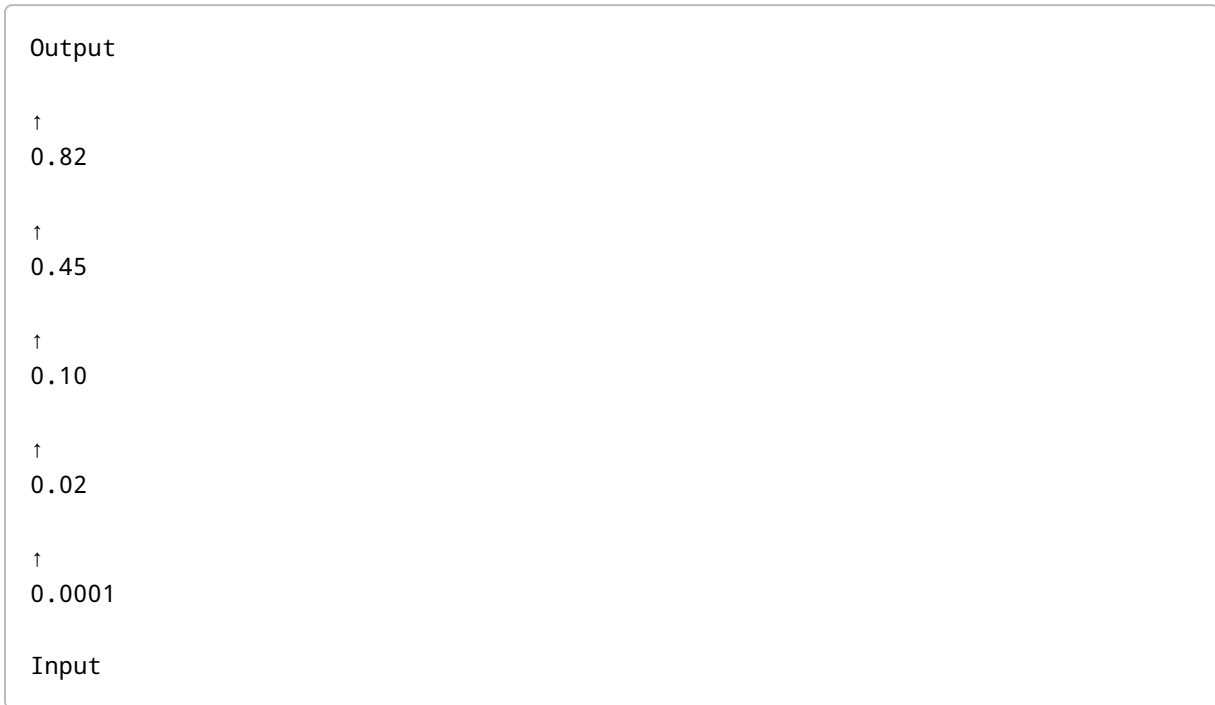
- Training becomes extremely slow.
- GPUs remain underutilized.
- Scaling to internet-sized corpora is impractical.

One of the Transformer's greatest innovations was removing this sequential bottleneck.

4. The Vanishing Gradient Problem

Neural networks learn using **backpropagation**, where gradients flow backward through the network to update weights.

In long RNNs, gradients may become extremely small as they propagate through many time steps.



Eventually, the gradient becomes so small that earlier layers stop learning.

This is known as the **Vanishing Gradient Problem**.

Consequences

- Early words are effectively forgotten.
- Long-term dependencies are lost.
- Training stalls.

For example:

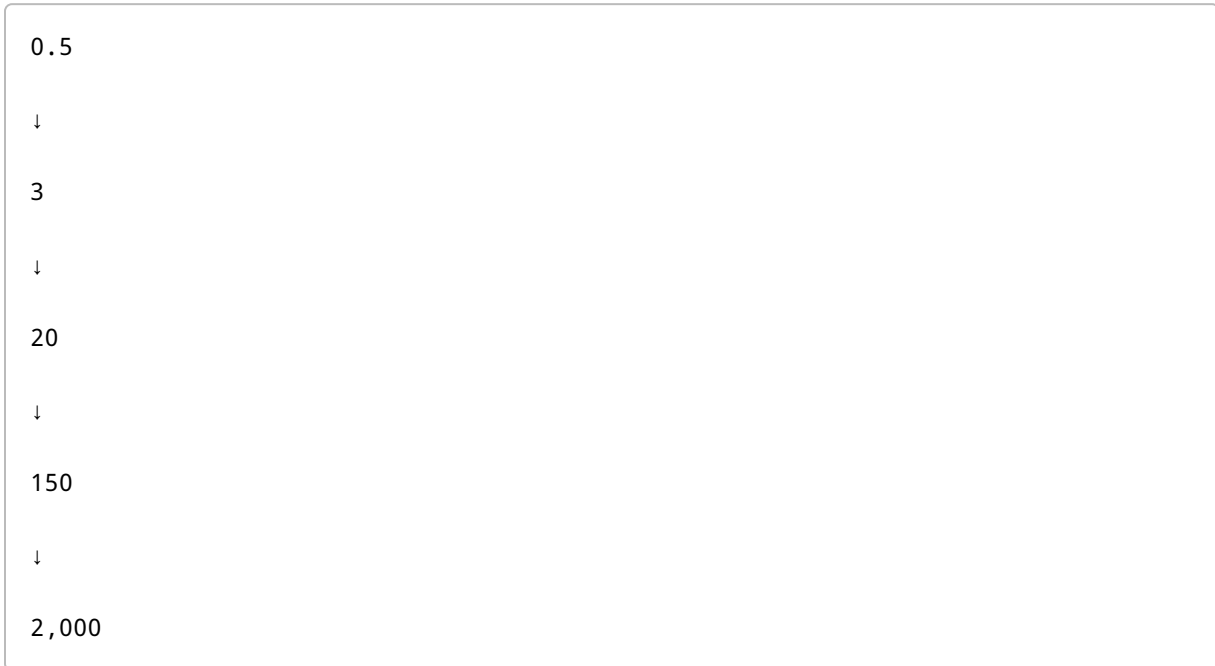
"John, who moved to London after completing his master's degree and working in Paris for five years, finally opened **his** company."

By the time the model reaches "his", it may have lost the connection to "John".

5. The Exploding Gradient Problem

The opposite issue can also occur.

Instead of shrinking, gradients may grow exponentially.



Large gradients produce unstable updates, causing training to diverge.

Common mitigation strategies included:

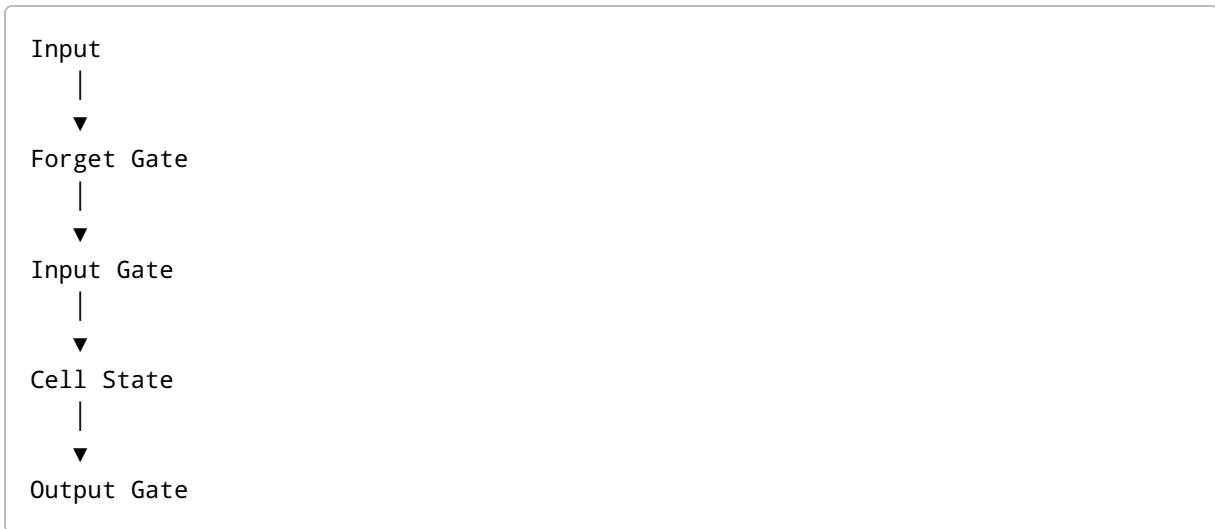
- Gradient clipping
- Careful weight initialization
- Learning-rate scheduling

While helpful, these techniques did not eliminate the underlying limitations.

6. Long Short-Term Memory (LSTM)

Introduced in 1997, LSTMs were designed to preserve information over longer sequences.

Instead of relying on a single hidden state, an LSTM introduces a **cell state** that acts like a long-term memory.



The three gates decide:

- What information should be forgotten.
- What new information should be stored.
- What information should be exposed as output.

This dramatically improved performance on many NLP tasks.

7. Why LSTMs Still Fell Short

Although LSTMs solved part of the memory problem, they still inherited several limitations:

Limitation	Why It Matters
Sequential computation	Prevents efficient parallel training.
Long documents	Information degrades over very long sequences.
High latency	Slow inference for lengthy inputs.
Scaling	Difficult to train on web-scale datasets.

Consider a legal contract with 300 pages.

Even an LSTM struggles to maintain relationships between clauses that are hundreds or thousands of tokens apart.

8. The Real-World Bottleneck

Imagine building a customer-support assistant.

Knowledge Base:

- 20 million support tickets
- 100,000 product manuals
- 5 TB of documentation

Training an LSTM-based system on this scale would be prohibitively slow, and reasoning across such long contexts would remain unreliable.

The industry needed an architecture that could:

- Process tokens in parallel.
- Capture long-range dependencies.
- Scale efficiently across many GPUs.
- Train on internet-scale corpora.

That architecture became the **Transformer**.

9. Key Takeaways

- RNNs process one token at a time, making training sequential.
 - Vanishing gradients prevent learning long-range relationships.
 - Exploding gradients make optimization unstable.
 - LSTMs improve memory with gating mechanisms but still process sequentially.
 - These limitations motivated the development of the Transformer architecture, which replaced recurrence with attention.
-

Production Notes

Why this matters for AI engineers

Modern LLMs such as GPT, Llama, Claude, Gemini, and Mistral do **not** use RNNs or LSTMs as their core architecture. Understanding their limitations helps explain why attention-based models became the standard and why they are so effective for Retrieval-Augmented Generation systems.

Interview Preparation

Q1. Why can't RNNs be parallelized effectively?

Answer: Each hidden state depends on the previous hidden state, creating a strict sequential dependency. This prevents simultaneous processing of tokens and limits GPU utilization.

Q2. What is the vanishing gradient problem?

Answer: During backpropagation, gradients can become exponentially smaller as they pass through many layers or time steps, preventing earlier layers from learning effectively and causing the model to forget long-range dependencies.

Q3. How do LSTMs improve upon standard RNNs?

Answer: LSTMs introduce a cell state and gating mechanisms (forget, input, and output gates) that regulate information flow, allowing important information to persist over longer sequences.

Q4. Why were LSTMs eventually replaced by Transformers?

Answer: Although LSTMs improved memory, they remained sequential, limiting training speed and scalability. Transformers process all tokens simultaneously using self-attention, enabling parallel computation and better modeling of long-range relationships.

Q5. Why is understanding RNN limitations relevant to RAG?

Answer: RAG systems rely on modern Transformer-based LLMs. Knowing why Transformers replaced earlier architectures provides insight into why they can efficiently reason over retrieved context and scale to enterprise workloads.

Exercises

1. Draw the computation flow of an RNN and identify the sequential dependency.
2. Explain the difference between vanishing and exploding gradients in your own words.
3. Describe how the forget gate in an LSTM helps preserve long-term information.
4. Imagine you need to process a 500-page document. Explain why an RNN or LSTM would struggle compared to a Transformer.